

LAPORAN PRAKTIKUM
POSTTEST 1
PEMROGRAMAN BERBASIS OBJEK



Disusun oleh:
Antung Hissyam (2509106092)
Kelas (C1 '25)

PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
SAMARINDA
2026

1. Pendahuluan

Program pada praktikum ini dibuat menggunakan bahasa Python dengan pendekatan Object Oriented Programming (OOP). Tema yang digunakan adalah manajemen pegawai pada Kebab Cendana. Program mengelola data pegawai, jadwal kerja, dan perhitungan gaji.

Pembuatan program mengacu pada tiga materi yang dipelajari, yaitu Class & Object, Atribut & Method, serta Encapsulation & Property. Program tidak menggunakan inheritance karena pada ketentuan tugas inheritance tidak diwajibkan.

Selain menjelaskan struktur program, laporan ini membandingkan kode dengan ketentuan tugas dan menggunakan hasil run program yang diberikan sebagai bukti pengujian.

2. Tujuan Program

1. Menerapkan konsep class dan object dalam Python.
2. Menerapkan atribut kelas dan atribut instance.
3. Menerapkan instance method, class method, dan static method.
4. Menerapkan encapsulation menggunakan atribut private.
5. Menerapkan getter dan setter menggunakan @property.
6. Menerapkan validasi agar data yang masuk sesuai aturan program.
7. Menguji program dengan dua objek untuk setiap class dan dengan data valid maupun tidak valid

3. Struktur Class dan Hubungan Antarclass

Program memiliki tiga class utama yang berdiri sendiri tetapi saling berinteraksi.

Class	Fungsi	Atribut utama	Method utama
Pegawai	Menyimpan identitas dan PIN pegawai.	nama, jabatan, __pin_absen	profil_pegawai(), ubah_nama_instansi(), cek_format_pin()
ShiftKerja	Menyimpan jadwal kerja pegawai.	pegawai, tanggal, __jenis_shift	info_jadwal(), tambah_pilihan_shift(), cek_ketersediaan_shift()
GajiPegawai	Menghitung gaji dan total pengeluaran.	pegawai, hari_hadir, __gaji_harian	cetak_slip_gaji(), akumulasi_pengeluaran(), validasi_nominal()

Hubungan antarclass terlihat saat objek Pegawai dikirim ke constructor ShiftKerja dan GajiPegawai. Dengan cara ini, jadwal dan gaji dapat dikaitkan dengan pegawai tertentu.

4. Penerapan Atribut

4.1 Atribut Kelas

Ketentuan meminta minimal tiga atribut kelas. Ketentuan ini terpenuhi pada ketiga class.

```
class Pegawai:
    nama_instansi = "Kebab Cendana"
    total_pegawai = 0
    batas_pegawai = 50
```

```
class ShiftKerja:
    total_shift_dibuat = 0
    pilihan_shift = ["Pagi", "Sore", "Malam"]
    jam_operasional = "08:00 - 24:00"
```

```
class GajiPegawai:
    total_pengeluaran = 0
    potongan_pajak = 0.05
    mata_uang = "IDR"
```

Atribut-atribut tersebut bersifat class attribute sehingga nilainya dimiliki bersama oleh objek pada class yang sama. Contohnya total_pegawai berubah setiap kali objek Pegawai dibuat.

4.2 Atribut Instance

Atribut instance dibuat di dalam `__init__()` menggunakan self. Nilainya diterima dari parameter sehingga setiap objek dapat memiliki data yang berbeda.

```
def __init__(self, nama, jabatan, pin):
    self.nama = nama
    self.jabatan = jabatan
    self.__pin_absen = None
    self.pin_absen = pin
```

Pada ShiftKerja digunakan self.pegawai dan self.tanggal. Pada GajiPegawai digunakan self.pegawai, self.hari_hadir, dan self.__gaji_harian.

4.3 Public dan Private Attribute

Atribut public dapat diakses langsung, misalnya self.nama, self.jabatan, self.pegawai, dan self.tanggal. Atribut private ditandai dua garis bawah, yaitu `__pin_absen`, `__jenis_shift`, dan `__gaji_harian`. Data tersebut tidak diubah secara langsung dari luar class, tetapi melalui property.

5. Penerapan Method

5.1 Instance Method

Instance method menerima self dan bekerja terhadap data milik objek. Program memiliki beberapa instance method.

```
def profil_pegawai(self):  
    print(f"Pegawai: {self.nama} | Jabatan: {self.jabatan}")
```

Method tersebut digunakan untuk menampilkan profil pegawai. Contoh lain adalah info_jadwal() dan cetak_slip_gaji().

5.2 Class Method

Class method menggunakan decorator @classmethod dan parameter cls. Method ini digunakan untuk mengubah atau memperbarui data yang dimiliki bersama oleh class.

```
@classmethod  
def ubah_nama_instansi(cls, nama_baru):  
    cls.nama_instansi = nama_baru
```

Pada ShiftKerja terdapat tambah_pilihan_shift(), sedangkan pada GajiPegawai terdapat akumulasi_pengeluaran().

5.3 Static Method

Static method menggunakan @staticmethod dan tidak membutuhkan self maupun cls. Pada program, method ini digunakan sebagai fungsi bantuan untuk validasi.

```
@staticmethod  
def cek_format_pin(pin):  
    return str(pin).isdigit() and len(str(pin)) >= 4
```

Dua static method lain adalah cek_ketersediaan_shift() dan validasi_nominal().

6. Encapsulation, Getter, Setter, dan Validasi

Encapsulation diterapkan dengan menyimpan data penting pada atribut private. Akses data dilakukan melalui property sehingga perubahan data dapat diperiksa terlebih dahulu.

6.1 Getter dan Setter PIN

```
@pin_absen.setter  
def pin_absen(self, pin_baru):  
    if Pegawai.cek_format_pin(pin_baru):  
        self.__pin_absen = pin_baru
```

```
else:
    raise ValueError("PIN harus berupa angka dan minimal 4 digit")
```

Getter menggunakan @property dan setter memakai nama property yang sama, yaitu pin_absen. Setter memanggil validasi PIN sebelum mengubah __pin_absen.

6.2 Setter Jenis Shift

```
@jenis_shift.setter
def jenis_shift(self, shift_baru):
    if ShiftKerja.cek_ketersediaan_shift(shift_baru, self.pilihan_shift):
        self.__jenis_shift = shift_baru
    else:
        raise ValueError("Jenis shift tidak terdaftar di sistem")
```

Jenis shift hanya dapat diubah jika terdapat di daftar pilihan shift. Hal ini mencegah data shift yang tidak sesuai masuk ke objek.

6.3 Setter Gaji

```
@staticmethod
def validasi_nominal(nominal):
    return isinstance(nominal, (int, float)) and nominal > 0

@gaji_harian.setter
def gaji_harian(self, nominal):
    if self.validasi_nominal(nominal):
        self.__gaji_harian = nominal
    else:
        raise ValueError("Nominal gaji tidak boleh nol atau minus")
```

Nominal gaji harus berupa int atau float dan nilainya harus lebih dari nol. Jika tidak memenuhi syarat, setter membatalkan perubahan dan menghasilkan ValueError.

7. Alur Program pada Main Code

Pengujian program berada di dalam if __name__ == "__main__":. Bagian ini digunakan untuk membuat objek dan memperagakan fungsi program.

7.1 Membuat Dua Objek Pegawai

```
pegawai1 = Pegawai("Antung", "Kasir", "1024")
pegawai2 = Pegawai("Dimas", "Dapur", "2048")
```

Dua objek Pegawai berhasil dibuat. Masing-masing mempunyai nama, jabatan, dan PIN sendiri. Saat objek dibuat, total_pegawai bertambah sehingga nilainya menjadi 2.

7.2 Mengubah Atribut Kelas dan Menampilkan Profil

```
Pegawai.ubah_nama_instansi("Kebab Cendana Pusat")
print(Pegawai.nama_instansi)
pegawai1.profil_pegawai()
pegawai2.profil_pegawai()
```

Class method digunakan untuk mengubah nama instansi. Setelah itu instance method `profil_pegawai()` digunakan untuk menampilkan data kedua pegawai.

7.3 Membuat Dua Jadwal Kerja

```
shift1 = ShiftKerja(pegawai1, "23-10-2026", "Pagi")
shift2 = ShiftKerja(pegawai2, "23-10-2026", "Sore")
ShiftKerja.tambah_pilihan_shift("Tengah Malam")
```

Dibuat dua objek `ShiftKerja` yang masing-masing terhubung dengan objek `Pegawai`. Class method kemudian menambahkan pilihan shift baru bernama `Tengah Malam`.

7.4 Membuat Dua Data Gaji

```
shift1 = ShiftKerja(pegawai1, "23-10-2026", "Pagi")
shift2 = ShiftKerja(pegawai2, "23-10-2026", "Sore")
ShiftKerja.tambah_pilihan_shift("Tengah Malam")
```

Setiap data gaji menyimpan pegawai, jumlah hari hadir, dan gaji harian. Method `cetak_slip_gaji()` menghitung gaji kotor, pajak lima persen, dan gaji bersih.

7.5 Perhitungan Gaji

Untuk Antung, 20 hari dikali Rp100.000 menghasilkan gaji kotor Rp2.000.000. Potongan pajak 5% sebesar Rp100.000 sehingga gaji bersih Rp1.900.000.

Untuk Dimas, 15 hari dikali Rp120.000 menghasilkan gaji kotor Rp1.800.000. Potongan pajak 5% sebesar Rp90.000 sehingga gaji bersih Rp1.710.000. Total pengeluaran gaji menjadi Rp3.610.000.

8. Hasil Run Program

Berikut adalah hasil run yang diberikan dari terminal. Output ini digunakan sebagai bukti bahwa program berjalan tanpa error dan seluruh pengujian yang dibuat menghasilkan keluaran sesuai logika program.

```
PS E:\Coding\praktikum-pbo\posttest\posttest1> py a.py
Kebab Cendana Pusat
Pegawai: Antung | Jabatan: Kasir
Pegawai: Dimas | Jabatan: Dapur
Total pegawai terdaftar: 2

Jadwal Antung pada 23-10-2026: Shift Pagi
Jadwal Dimas pada 23-10-2026: Shift Sore
Total jadwal dibuat: 2

Gaji Antung (20 hari): IDR 1900000.0
Gaji Dimas (15 hari): IDR 1710000.0
Total pengeluaran gaji: IDR 3610000.0

Uji validasi data salah:
Gagal ubah PIN Antung: PIN harus berupa angka dan minimal 4 digit
Gagal ubah shift Dimas: Jenis shift tidak terdaftar di sistem
Gagal ubah gaji Antung: Nominal gaji tidak boleh nol atau minus

Uji validasi data benar:
Data Antung dan Dimas berhasil diubah.
```

9. Pengujian Program

Pengujian dibagi menjadi dua, yaitu pengujian data tidak valid dan pengujian data valid.

Uji	Input	Hasil pada terminal	Keterangan
PIN tidak valid	"12"	Gagal ubah PIN Antung	Ditolak karena kurang dari 4 digit.
Shift tidak valid	"Subuh"	Gagal ubah shift Dimas	Ditolak karena tidak ada dalam daftar shift.
Gaji tidak valid	-50000	Gagal ubah gaji Antung	Ditolak karena nilainya tidak lebih dari 0.
PIN valid	"9999"	Data berhasil diubah	Memenuhi format PIN.
Shift valid	"Tengah Malam"	Data berhasil diubah	Shift sudah ditambahkan ke daftar.
Gaji valid	150000	Data berhasil diubah	Nominal lebih dari 0.

10. Kesimpulan

Program Manajemen Pegawai Kebab Cendana telah menerapkan konsep utama OOP yang diminta pada tugas, yaitu tiga class, atribut kelas dan instance, public dan private attribute, instance method, class method, static method, encapsulation, property, setter, dan validasi data.

Hasil run menunjukkan program dapat membuat dua objek untuk setiap class, menampilkan data pegawai dan jadwal, menghitung gaji, serta menolak data yang tidak sesuai aturan. Dengan menambahkan demonstrasi langsung static method pada bagian main.

Lampiran A. Kode Program yang Digunakan

```
class Pegawai:
    nama_instansi = "Kebab Cendana"
    total_pegawai = 0
    batas_pegawai = 50

    def __init__(self, nama, jabatan, pin):
        self.nama = nama
        self.jabatan = jabatan
        self.__pin_absen = None
        self.pin_absen = pin
        Pegawai.total_pegawai += 1

    @property
    def pin_absen(self):
        return self.__pin_absen

    @pin_absen.setter
    def pin_absen(self, pin_baru):
        if Pegawai.cek_format_pin(pin_baru):
            self.__pin_absen = pin_baru
        else:
            raise ValueError("PIN harus berupa angka dan minimal 4 digit")

    def profil_pegawai(self):
        print(f"Pegawai: {self.nama} | Jabatan: {self.jabatan}")

    @classmethod
    def ubah_nama_instansi(cls, nama_baru):
        cls.nama_instansi = nama_baru

    @staticmethod
    def cek_format_pin(pin):
        return str(pin).isdigit() and len(str(pin)) >= 4

class ShiftKerja:
    total_shift_dibuat = 0
    pilihan_shift = ["Pagi", "Sore", "Malam"]
    jam_operasional = "08:00 - 24:00"

    def __init__(self, pegawai, tanggal, jenis_shift):
```

```

        self.pegawai = pegawai
        self.tanggal = tanggal
        self.__jenis_shift = None
        self.jenis_shift = jenis_shift
        ShiftKerja.total_shift_dibuat += 1

    @property
    def jenis_shift(self):
        return self.__jenis_shift

    @jenis_shift.setter
    def jenis_shift(self, shift_baru):
        if ShiftKerja.cek_ketersediaan_shift(shift_baru, self.pilihan_shift):
            self.__jenis_shift = shift_baru
        else:
            raise ValueError("Jenis shift tidak terdaftar di sistem")

    def info_jadwal(self):
        print(f"Jadwal {self.pegawai.nama} pada {self.tanggal}: Shift {self.__jenis_shift}")

    @classmethod
    def tambah_pilihan_shift(cls, shift_baru):
        if shift_baru not in cls.pilihan_shift:
            cls.pilihan_shift.append(shift_baru)

    @staticmethod
    def cek_ketersediaan_shift(shift, daftar_shift):
        return shift in daftar_shift


class GajiPegawai:
    total_pengeluaran = 0
    potongan_pajak = 0.05
    mata_uang = "IDR"

    def __init__(self, pegawai, hari_hadir, gaji_harian):
        self.pegawai = pegawai
        self.hari_hadir = hari_hadir
        self.__gaji_harian = None
        self.gaji_harian = gaji_harian

    @property
    def gaji_harian(self):
        return self.__gaji_harian

    @gaji_harian.setter
    def gaji_harian(self, nominal):
        if self.validasi_nominal(nominal):
            self.__gaji_harian = nominal
        else:
            raise ValueError("Nominal gaji tidak boleh nol atau minus")

    def cetak_slip_gaji(self):
        kotor = self.hari_hadir * self.__gaji_harian
        pajak = kotor * self.potongan_pajak
        bersih = kotor - pajak

```

```

        GajiPegawai.akumulasi_pengeluaran(bersih)
        print(f"Gaji {self.pegawai.nama} ({self.hari_hadir} hari): {self.mata_uang}
{bersih}")

    @classmethod
    def akumulasi_pengeluaran(cls, nominal):
        cls.total_pengeluaran += nominal

    @staticmethod
    def validasi_nominal(nominal):
        return isinstance(nominal, (int, float)) and nominal > 0

if __name__ == "__main__":
    pegawai1 = Pegawai("Antung", "Kasir", "1024")
    pegawai2 = Pegawai("Dimas", "Dapur", "2048")
    Pegawai.ubah_nama_instansi("Kebab Cendana Pusat")
    print(Pegawai.nama_instansi)
    pegawai1.profil_pegawai()
    pegawai2.profil_pegawai()
    print(f"Total pegawai terdaftar: {Pegawai.total_pegawai}\n")

    shift1 = ShiftKerja(pegawai1, "23-10-2026", "Pagi")
    shift2 = ShiftKerja(pegawai2, "23-10-2026", "Sore")
    ShiftKerja.tambah_pilihan_shift("Tengah Malam")
    shift1.info_jadwal()
    shift2.info_jadwal()
    print(f"Total jadwal dibuat: {ShiftKerja.total_shift_dibuat}\n")

    gaji1 = GajiPegawai(pegawai1, 20, 100000)
    gaji2 = GajiPegawai(pegawai2, 15, 120000)
    gaji1.cetak_slip_gaji()
    gaji2.cetak_slip_gaji()
    print(f"Total pengeluaran gaji: {GajiPegawai.mata_uang}
{GajiPegawai.total_pengeluaran}\n")

    print("Uji validasi data salah:")
    try:
        pegawai1.pin_absen = "12"
    except ValueError as e:
        print(f"Gagal ubah PIN Antung: {e}")
    try:
        shift2.jenis_shift = "Subuh"
    except ValueError as e:
        print(f"Gagal ubah shift Dimas: {e}")
    try:
        gaji1.gaji_harian = -50000
    except ValueError as e:
        print(f"Gagal ubah gaji Antung: {e}")

    print("\nUji validasi data benar:")
    pegawai1.pin_absen = "9999"
    shift2.jenis_shift = "Tengah Malam"
    gaji1.gaji_harian = 150000
    print("Data Antung dan Dimas berhasil diubah.")

```